

Week 4 Module

By Mohit Sharma, 2404921540131

Week 4 Module

→ An algorithm is simply a step-by-step procedure to solve a problem

example → Find largest element in array

- Assume first element is largest
- Compare with every other element
- Update largest whenever needed
- Print largest

Time complexity

→ How Running time increases as input size increases

example → $n=5$; `print("Hello");`

$O(1)$ [constant]

Space complexity

→ How much extra memory does the program [algorithm] uses

example → `int a;`

$O(1)$ [constant]

Asymptotic notations - algorithm

Big O (O)

- Maximum time (upper bound)
- Worst case growth

Omega (Ω)

- Minimum time (lower bound)
- Best case growth

Theta (Θ)

- Exact Growth (tight + upper/lower b.)
- Average case

Nested loops $\Rightarrow$

loops : $\text{for}(i=0; i < n; i++) \{ \}$

$\Rightarrow O(n)$

nested loops : $\text{for}(i=0; i < n; i++) \{$
 $\text{for}(j=0; j < n; j++) \{$
 $\}$
 $\}$

$\Rightarrow O(n^2)$

Section-A

Module-1

Module-1

STAR Patterns, Number patterns

→ Time complexity of nested loops:→

```
for (i=0 ; i<n ; i++) {  
    for (j=0 ; j<n ; j++) {  
        }  
    }
```

these run for $(n \times n) \Rightarrow n^2$ iterations
hence

$$\boxed{\text{Time} \Rightarrow O(n^2)}$$

→ None of pattern problems require any space hence,

$$\boxed{\text{space} \Rightarrow O(1)}$$

No further optimization is possible because algorithm must print every required character or number. Since output itself contains approximately $O(n^2)$ elements, the time complexity is already optimal.

Module -2# linear search $\Rightarrow$ Time $\Rightarrow O(n)$ space $\Rightarrow O(1)$

Explanation $\Rightarrow$ algorithm checks each element of array one by one until target element is found or array ends.

worst case, all array indices are checked

best-case, first index has the target element

Optimization $\Rightarrow$ Binary search $\Rightarrow$ Time $\Rightarrow O(\log n)$ space $\Rightarrow O(1)$

Explanation $\Rightarrow$ Binary search repeatedly divides search space into two halves by comparing target with middle element

Bubble sort $\Rightarrow$

Time $\Rightarrow O(n^2)$
space $\Rightarrow O(1)$

Optimized approach - optimized bubble sort uses a swap flag to stop early if no swaps occur during a pass, improving the best-case time complexity to $O(n)$.

Insertion sort $\Rightarrow$

Time $\Rightarrow O(n^2)$
space $\Rightarrow O(1)$

Optimization - Insertion sort is efficient for small or nearly sorted arrays, where its best-case time complexity becomes $O(n)$. For large datasets, Merge / Quick sort are better.

Selection sort $\Rightarrow$

Time $\Rightarrow O(n^2)$
space $\Rightarrow O(1)$

Optimization - for large data, merge / heap sort perform better.

Module-3

module-3

Merge sort $\Rightarrow$

Time $\Rightarrow O(n \log n)$
space $\Rightarrow O(n)$

optimization - merge sort has optimal $O(n \log n)$ time comp for comparison based sorting. However its space usage can be reduced using advanced, in-place merge techniques.

Quick sort $\Rightarrow$

Time $\Rightarrow O(n \log n)$, worst $\Rightarrow O(n^2)$
space $\Rightarrow O(\log n)$

optimization $\Rightarrow$ using random pivot or median of three pivot selection greatly reduces chances of worst case.

Heap sort $\Rightarrow$

Time $\Rightarrow O(n \log n)$
space $\Rightarrow O(1)$

optimization $\Rightarrow$ Heap sort is already an efficient in-place sorting algo with $O(n \log n)$ time complexity.

Section-B

Problem:

Given a positive integer N, print all the numbers from 1 to N in increasing order, separated by a single space.

Your task is to write a program that prints the required sequence.

After completing the program, analyze your solution by calculating its Time Complexity and Space Complexity, and explain why your answer is correct.

Input Format

A single integer N.

Output Format

Print all integers from 1 to N in increasing order separated by spaces.

Constraints

$$1 \leq N \leq 10^5$$

Code:

```
Week 3 Module.cpp X
Week 3 Module.cpp
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int N;
6      cin >> N;
7
8      for (int i = 1; i <= N; i++) {
9          cout << i << " ";
10     }
11
12     return 0;
13 }
```

Analysis:-

Section-B

① Time $\Rightarrow O(n)$
space $\Rightarrow O(1)$

Optimized approach -

No further optimization is possible because every number from 1 to N must be printed exactly once. Since the output itself contains N numbers, the algorithm already has minimum possible time complexity of $O(n)$.

Problem:

Given a positive integer N , find the sum of the first N natural numbers using a loop.

Your task is to write a program that calculates and prints the required sum.

After completing the program, analyze your solution by calculating its Time Complexity and Space Complexity, and explain your answer.

Input Format

A single integer N .

Output Format

Print the sum of the first N natural numbers.

Constraints

$$1 \leq N \leq 10^7$$

Code:

```
Week 3 Module.cpp X
Week 3 Module.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int N;
6      cin >> N;
7
8      int sum = 0;
9
10     for (int i = 1; i <= N; i++) {
11         sum += i;
12     }
13
14     cout << sum;
15
16     return 0;
17 }
```

Analysis:

(2) Time $\Rightarrow O(n)$
space $\Rightarrow O(1)$

Explanation / optimization $\Rightarrow$

Optimized approach uses mathematical formula

$$\text{Sum} = \frac{N(N+1)}{2}$$

Optimized Time comp $\Rightarrow O(1)$
Optimized space comp $\Rightarrow O(1)$

Problem:

Given two integers A and B, calculate the value of A^B (A raised to the power B) using a loop.

Your task is to write a program that computes the result and prints it.

Once your program is complete, analyze your solution by calculating its Time Complexity and Space Complexity, and explain why your answer is correct.

Input Format

Two integers A and B separated by a space.

Output Format

Print the value of A^B .

Constraints

$$1 \leq A \leq 100$$

$$0 \leq B \leq 20$$

Code:

```
Week 3 Module.cpp X
Week 3 Module.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int A, B;
6      cin >> A >> B;
7
8      int result = 1;
9
10     for (int i = 1; i <= B; i++) {
11         result *= A;
12     }
13
14     cout << result;
15
16     return 0;
17 }
```

Analysis:

Date: _____
Page: _____

③ Time $\Rightarrow O(B)$
space $\Rightarrow O(1)$

Exp/optimized approach $\Rightarrow$

An optimized approach is binary exponentiation (Fast exponentiation), which repeatedly squares the base and halves exponent.

This reduces time complexity from $O(B)$ to $O(\log B)$ while keeping space complexity $O(1)$.